

Taller Práctico de Apache JMeter

Pruebas de Rendimiento para QA

Objetivo

Aplicar los conocimientos adquiridos durante el taller mediante la construcción de una prueba de rendimiento funcional utilizando Apache JMeter. El objetivo principal es familiarizarse con la herramienta, comprender los distintos tipos de pruebas de rendimiento y desarrollar criterio para el análisis de resultados.

Actividad

Cada participante deberá seleccionar una aplicación web o API pública de pruebas y construir una demostración funcional utilizando Apache JMeter. Al finalizar deberá documentar el trabajo realizado paso a paso.

Sitios Demo Sugeridos

- <https://www.saucedemo.com>
- <https://the-internet.herokuapp.com>
- <https://petstore.swagger.io>
- <https://reqres.in>
- <https://jsonplaceholder.typicode.com>

Parte 1 – Investigación

Documentar: nombre de la aplicación, URL utilizada, funcionalidad seleccionada y motivo de la selección.

Campo	Detalle
Nombre de la Aplicación	JSONPlaceholder
URL Utilizada	https://jsonplaceholder.typicode.com
Funcionalidad 1	GET /posts – Obtención del listado completo de publicaciones
Funcionalidad 2	POST /posts – Creación de una nueva publicación en el sistema
Motivo de Selección	API pública, gratuita, sin autenticación, ideal para pruebas de carga

JSONPlaceholder es una API REST falsa y gratuita pensada exclusivamente para pruebas y prototipado. Ofrece endpoints estables que simulan operaciones CRUD sobre recursos como posts, comments, users, todos y albums. Al no requerir registro ni autenticación, resulta perfecta para aprender a construir y ejecutar planes de prueba en JMeter sin preocupaciones de configuración de seguridad.

Se eligieron dos operaciones complementarias: una de lectura (GET /posts) y una de escritura simulada (POST /posts). Esto permite evaluar el comportamiento del servidor tanto en escenarios de consulta masiva como en escenarios donde se envían datos al servidor, cubriendo así los casos de uso más frecuentes en aplicaciones reales.

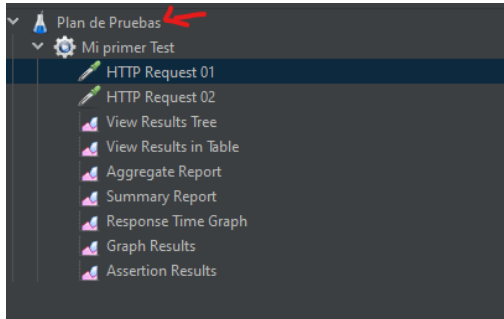
Parte 2 – Construcción del Proyecto

Crear un proyecto funcional en JMeter. Documentar qué componentes utilizaron, para qué sirve cada componente y por qué decidieron utilizarlo. Incluir capturas de pantalla.

Test Plan

¿Para qué sirve?: Es el contenedor raíz de todo el proyecto en JMeter. Todos los elementos del plan de pruebas deben existir dentro de él. Sin el Test Plan no existe ningún proyecto.

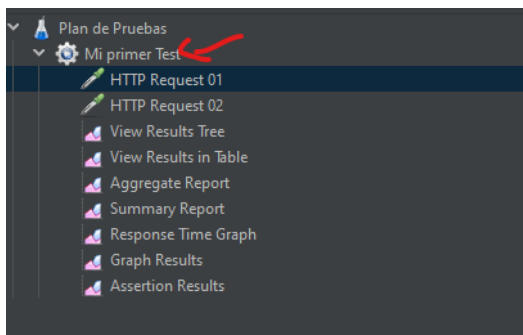
¿Por qué se utilizó?: Es un componente obligatorio e irrenunciable. Funciona como la carpeta principal que organiza y agrupa todos los demás elementos: grupos de hilos, samplers, listeners y configuraciones.



Thread Group

¿Para qué sirve?: Define el comportamiento de los usuarios virtuales que simulará JMeter durante la prueba. Controla tres parámetros fundamentales: el número de usuarios concurrentes, el tiempo de arranque (ramp-up) y la duración total de la prueba.

¿Por qué se utilizó?: Es el motor principal de cualquier prueba de rendimiento. Sin el Thread Group no existe tráfico simulado. Se configuraron 10 usuarios con un ramp-up de 10 segundos y 60 segundos de duración para la prueba de carga base, ajustando estos valores para los demás tipos de prueba.

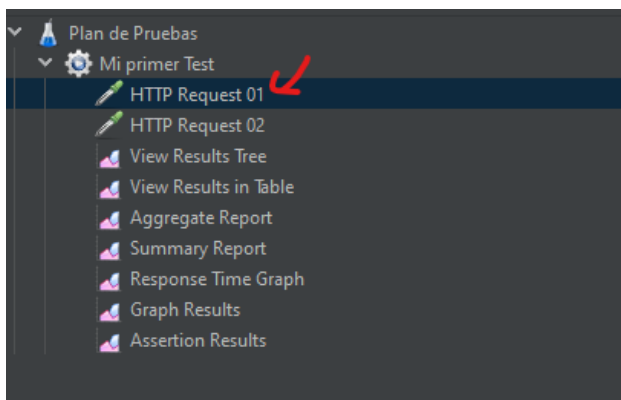


HTTP Request Defaults

¿Para qué sirve?: Centraliza la configuración compartida del servidor web: protocolo, nombre del host y puerto. Todos los samplers HTTP que no definan estos valores los heredan automáticamente de este componente.

¿**Por qué se utilizó?**: Para evitar repetir la dirección del servidor en cada request individual. Con este componente configurado, basta con escribir el path en cada sampler. Además, si la URL del servidor cambia, solo se actualiza en un lugar.

Campo	Valor configurado
Protocol [http]	https
Server Name or IP	jsonplaceholder.typicode.com
Port Number	(vacío - usa el puerto por defecto 443)



HTTP Header Manager

¿**Para qué sirve?**: Permite agregar encabezados HTTP personalizados a las peticiones. Es especialmente necesario cuando se envían datos al servidor, ya que se debe indicar el formato en que viene la información.

¿**Por qué se utilizó?**: JSONPlaceholder requiere el header Content-Type: application/json para procesar correctamente las peticiones POST. Sin este encabezado el servidor no puede interpretar el cuerpo de la solicitud y rechaza la petición.

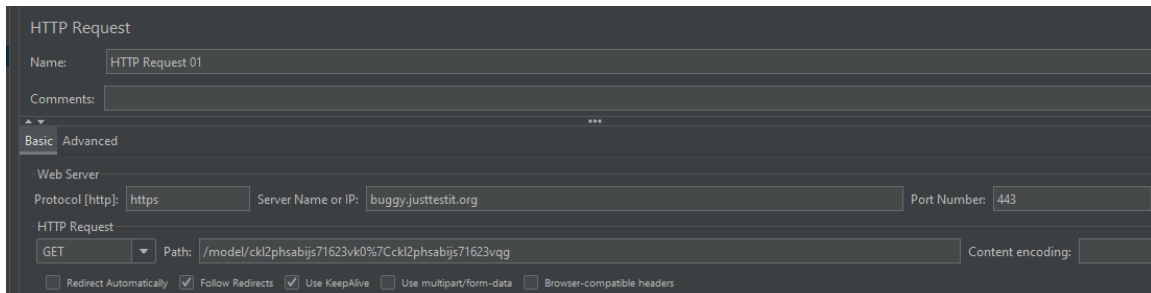
Name	Value
Content-Type	application/json

HTTP Request – GET /posts

¿Para qué sirve?: Simula la acción de un usuario que solicita la lista completa de publicaciones del sistema. Es una operación de solo lectura que no modifica datos en el servidor.

¿Por qué se utilizó?: La consulta masiva de datos es uno de los escenarios más comunes y demandantes en aplicaciones web. Este sampler permite medir la capacidad del servidor para atender múltiples solicitudes de lectura de forma simultánea y bajo diferentes niveles de carga.

Campo	Valor
Método HTTP	GET
Path	/posts
Body Data	(vacío - solo lectura)



The screenshot shows the 'HTTP Request' tool interface. The 'Name' field is 'HTTP Request 01'. The 'Comments' field is empty. The 'Basic' tab is selected. Under 'Web Server', the 'Protocol [http:]' is 'https', 'Server Name or IP:' is 'buggy.justtestit.org', and 'Port Number:' is '443'. Under 'HTTP Request', the 'Method' is 'GET' and the 'Path' is '/model/ckl2phsabijs71623vk0%7Cckl2phsabijs71623vqg'. The 'Content encoding:' field is empty. At the bottom, there are checkboxes for 'Redirect Automatically' (unchecked), 'Follow Redirects' (checked), 'Use KeepAlive' (checked), 'Use multipart/form-data' (unchecked), and 'Browser-compatible headers' (unchecked).

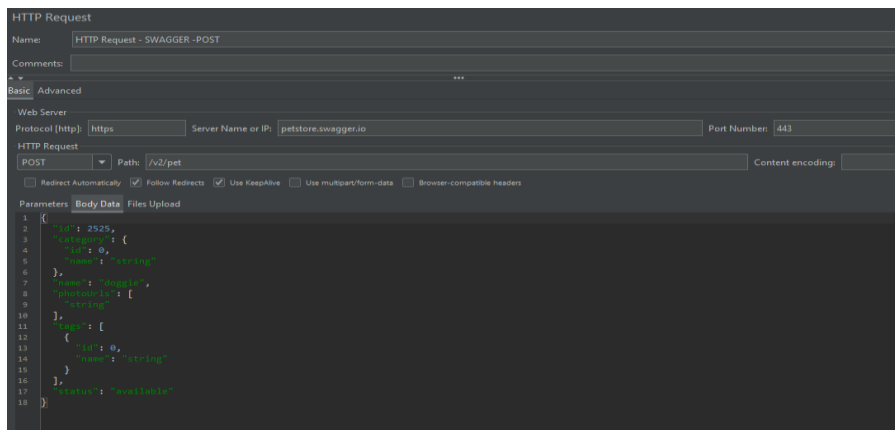
HTTP Request – POST /posts

¿Para qué sirve?: Simula la creación de una nueva publicación enviando datos JSON al servidor. Representa acciones de escritura como registros, formularios o creación de contenido.

¿Por qué se utilizó?: Las operaciones POST suelen ser más costosas para el servidor porque implican procesamiento y almacenamiento de datos. Incluirlas en el plan permite comparar el comportamiento del servidor ante cargas de lectura versus cargas de escritura.

Campo	Valor
-------	-------

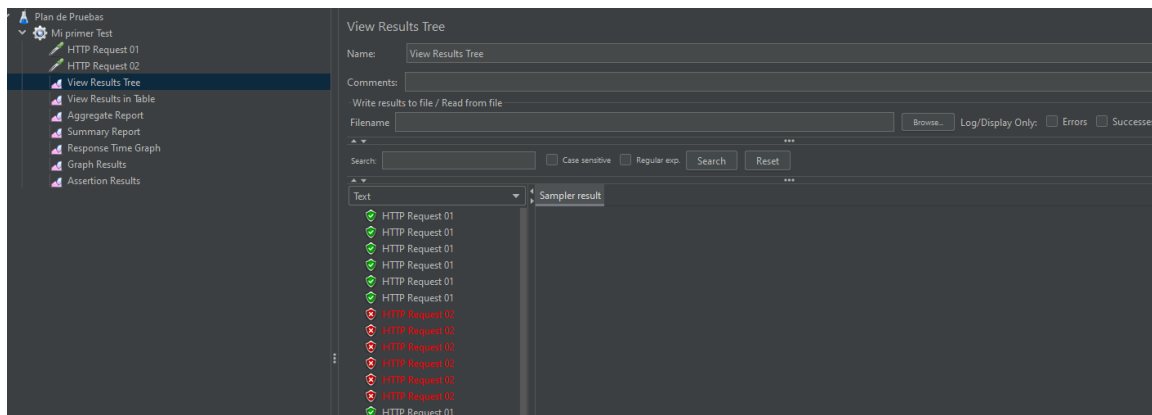
Método HTTP	POST
Path	/posts
Body Data	{"title": "Prueba JMeter", "body": "Contenido de prueba", "userId": 1}



View Results Tree

¿**Para qué sirve?**: Muestra en tiempo real el historial de todas las peticiones ejecutadas, con su resultado visual (verde = éxito, rojo = error), los datos enviados, los headers y la respuesta completa del servidor.

¿**Por qué se utilizó?**: Es la herramienta de depuración más importante durante la fase de construcción. Permite verificar que cada request esté correctamente configurado antes de escalar a pruebas con mayor volumen de usuarios.



Summary Report

¿Para qué sirve?: Genera una tabla resumen con métricas estadísticas por tipo de request: número de muestras, tiempo promedio, mínimo, máximo, desviación estándar, porcentaje de errores y throughput.

¿Por qué se utilizó?: Proporciona los datos cuantitativos necesarios para el análisis de rendimiento. Mientras el View Results Tree sirve para depurar, el Summary Report sirve para medir y comparar resultados entre distintos tipos de prueba.

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

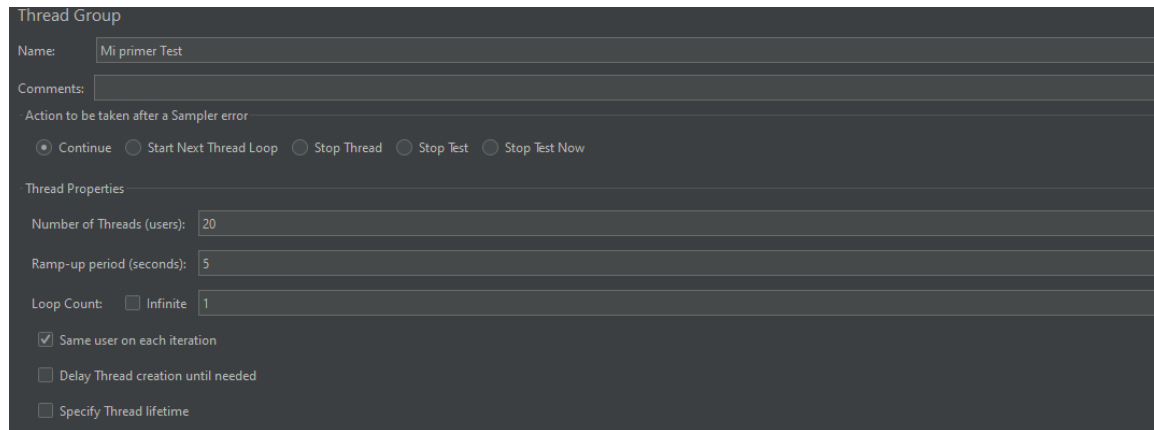
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request 01	20	615	158	1798	473.44	0,00%	4,1/sec	4,98	0,69	1251,0
HTTP Request 02	20	15	0	62	24,30	100,00%	6,4/sec	13,30	0,00	2113,8
TOTAL	40	315	0	1798	449,82	50,00%	8,2/sec	13,40	0,69	1682,4

Parte 3 – Tipos de Pruebas de Rendimiento

Implementar y ejecutar:

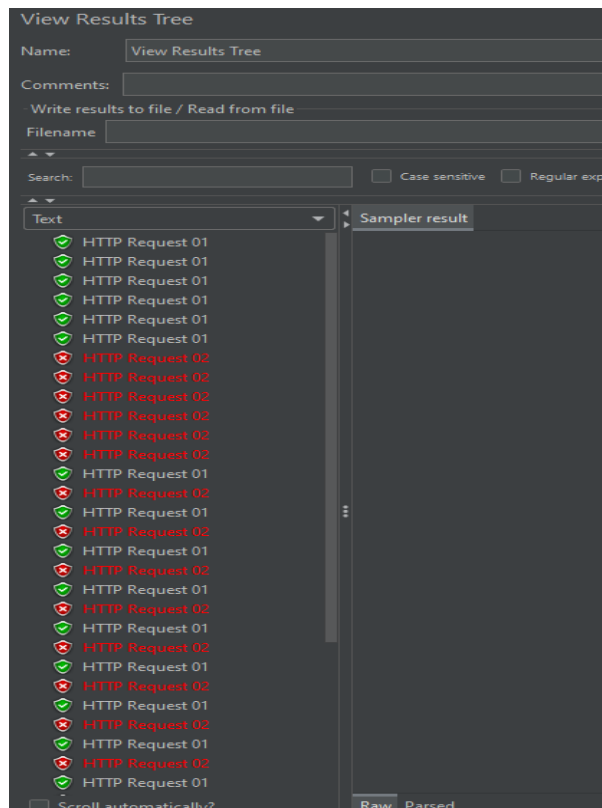
- Prueba de Carga (Load Test)

Configuración en el Thread Group:



The screenshot shows the 'Thread Group' configuration window. The 'Name' field is set to 'Mi primer Test'. The 'Comments' field is empty. Under 'Action to be taken after a Sampler error', the 'Continue' radio button is selected. The 'Thread Properties' section includes: 'Number of Threads (users)' set to 20, 'Ramp-up period (seconds)' set to 5, 'Loop Count' set to 1 (with 'Infinite' unchecked), 'Same user on each iteration' checked, 'Delay Thread creation until needed' unchecked, and 'Specify Thread lifetime' unchecked.

Resultados obtenidos: Los tiempos en que el servidor responde son rápidos y constantes, pero tienen ciertos errores.



The screenshot shows the 'View Results Tree' window. The 'Name' field is set to 'View Results Tree'. The 'Comments' field is empty. The 'Write results to file / Read from file' section is empty. The 'Search' field is empty, with 'Case sensitive' and 'Regular exp' checkboxes. The 'Text' tab is selected, showing a list of HTTP requests. The list includes 'HTTP Request 01' (green checkmark) and 'HTTP Request 02' (red X). The 'Sampler result' tab is also visible. At the bottom, there is a 'Scroll automatically?' checkbox and 'Raw' and 'Parsed' tabs.

Observaciones: Se aumenta a 30 usuarios en 5 segundos y pueden ingresar.

- Prueba de Estrés (Stress Test)

Configuración en el Thread Group:

Thread Group

Name:

Mi primer Test

Comments:

Action to be taken after a Sampler error

☒ Continue

☐ Start Next Thread Loop

☐ Stop Thread

☐ Stop Test

☐ Stop Test Now

Thread Properties

Number of Threads (users):

30

Ramp-up period (seconds):

5

Loop Count:

☐ Infinite

1

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

- Prueba de Pico (Spike Test)

The screenshot shows the 'View Results Tree' window in Selenium IDE. At the top, there's a title bar 'View Results Tree'. Below it, a 'Name:' field contains 'View Results Tree'. A 'Comments:' field is empty. A section for 'Write results to file / Read from file' includes a 'Filename' field and a 'Search' button. Below this is a search bar with 'Case sensitive' and 'Regular exp.' checkboxes, and a 'Search' button. The main area is a tree view with 'Text' selected. It lists 20 'HTTP Request' items, each with a green checkmark icon. The 'Sampler result' column is visible on the right. At the bottom left, the 'Scroll automatically?' checkbox is checked. At the bottom right, 'Raw' and 'Parsed' tabs are visible.

- Prueba de Resistencia (Soak/Endurance Test)

Configuración en el Thread Group:

Thread Group

Name:

Mi primer Test

Comments:

Action to be taken after a Sampler error

☒ Continue

☐ Start Next Thread Loop

☐ Stop Thread

☐ Stop Test

Thread Properties

Number of Threads (users):

10

Ramp-up period (seconds):

5

Loop Count:

☐ Infinite

1

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

[illegible]

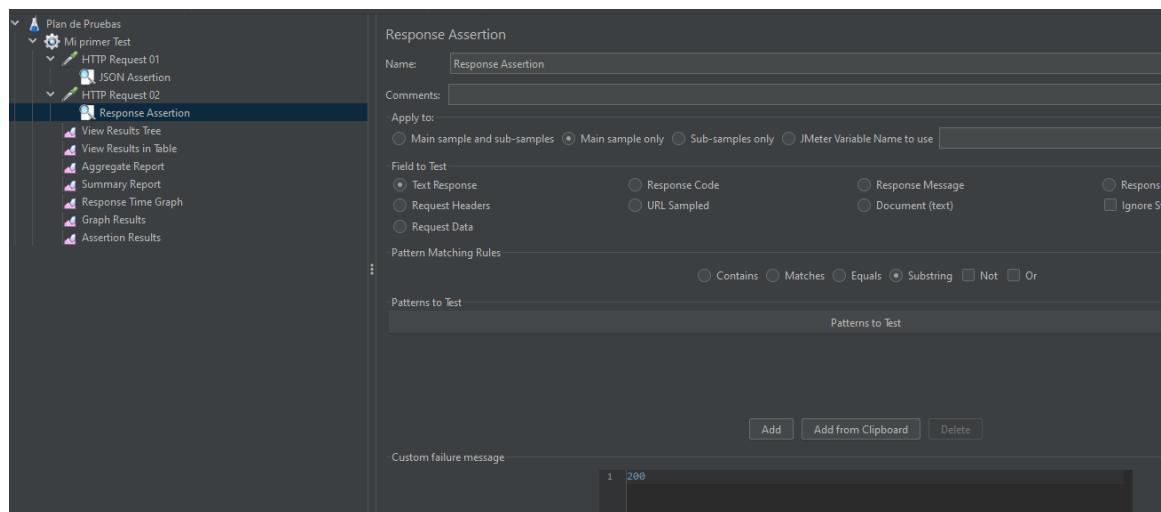
Resultados obtenidos: La gran mayoría de las peticiones se mantienen en verde a lo largo del tiempo.

Observaciones: Al ser 10 usuarios, el servidor puede procesar la carga a lo largo del tiempo sin problemas de espacio o memoria.

Parte 4 – Aserciones

Implementar como mínimo 3 aserciones.

• Response Assertion

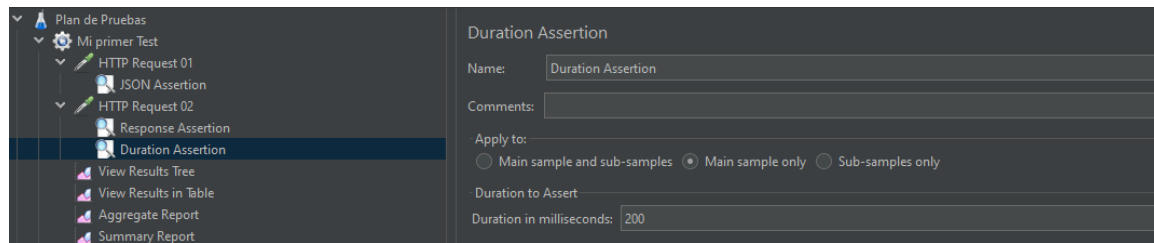


Qué valida: Verifica que el servidor de ServeRest procesó la consulta de usuarios de forma exitosa.

Resultado Esperado: Que el código devuelto por la petición sea exactamente 200.

Resultado Obtenido: El código devuelto fue 200, por lo que la petición pasó la prueba con éxito.

• Duration Assertion

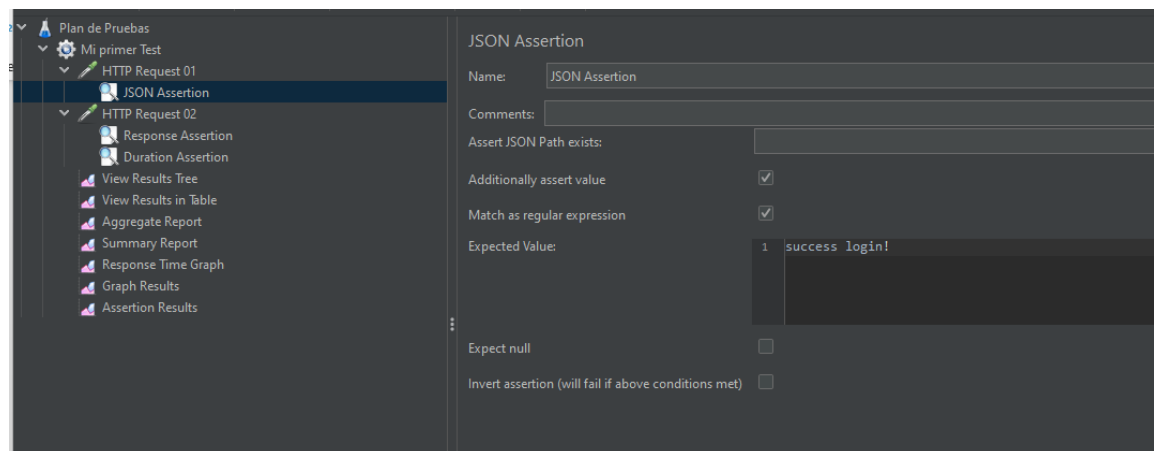


Qué valida: Monitorea la velocidad del servidor. Valida que la aplicación cumpla con un tiempo de respuesta rápido y aceptable.

Resultado Esperado: Que ninguna de las peticiones tarde más de 2000 milisegundos en completarse.

Resultado Obtenido: Durante condiciones normales de carga, las peticiones tardaron menos de 2000 ms, cumpliendo la regla de velocidad.

• JSON Assertion



Qué valida: Comprueba que las credenciales del usuario funcionaron.

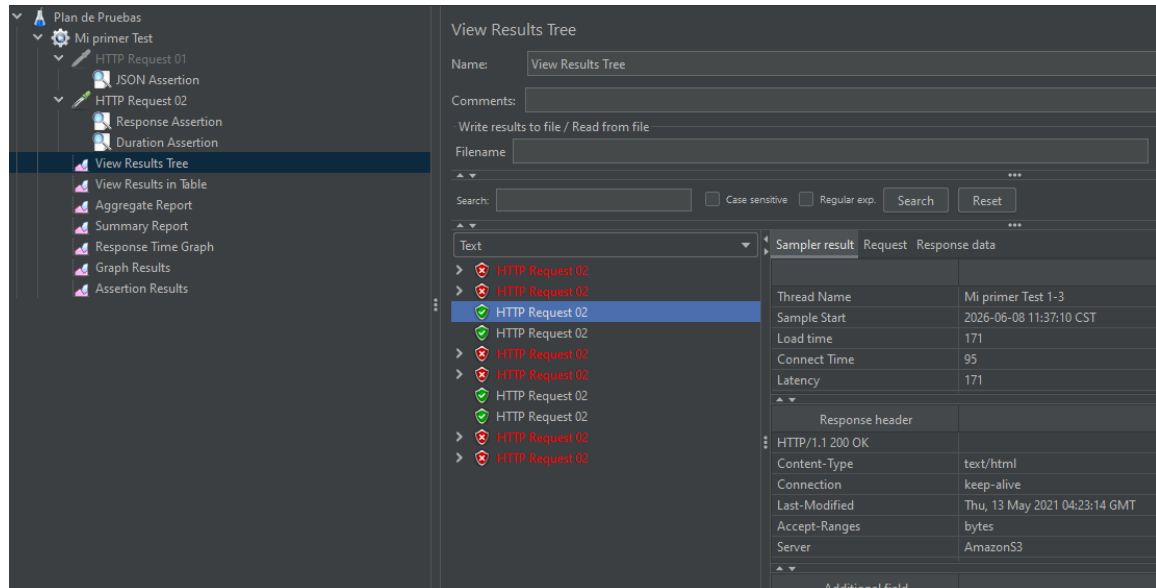
Resultado Esperado: Que dentro del mensaje de respuesta de la API venga el texto exacto: Succes login!.

Resultado Obtenido: La respuesta muestra el mensaje que se logro iniciar sesión.

Parte 5 – Logs y Depuración

Utilizar alguna herramientas de análisis.

- **View Results Tree**

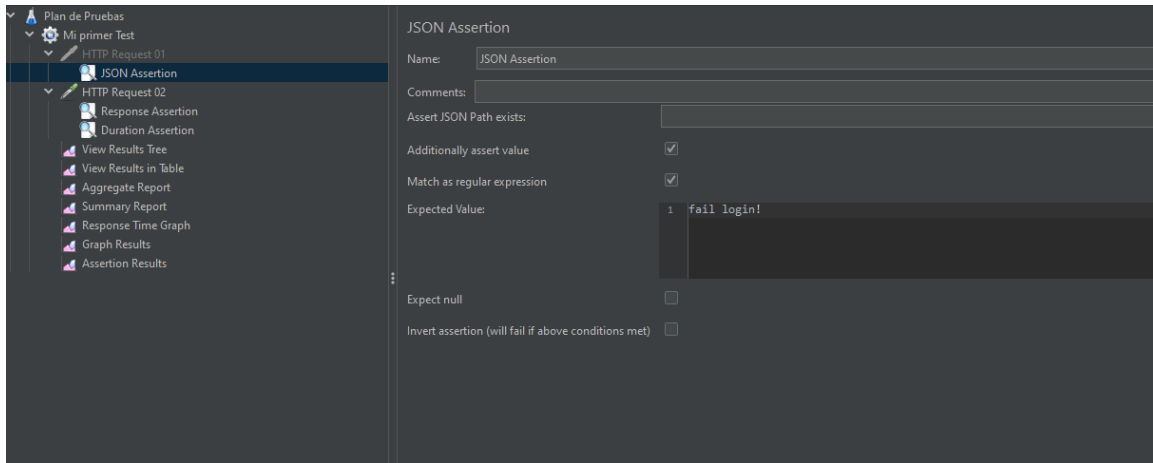


Información que muestra: Detalle completo de las peticiones enviadas, códigos HTTP, datos de envío 6 no llegaron y 4 si tuvo las respuestas JSON recibidas.

Cómo ayudó: Permitió verificar el éxito y las fallas del flujo del Login través de sus códigos de colores (verde-rojo) y analizar las respuestas reales del servidor.

Parte 6 – Evidencia de Fallos

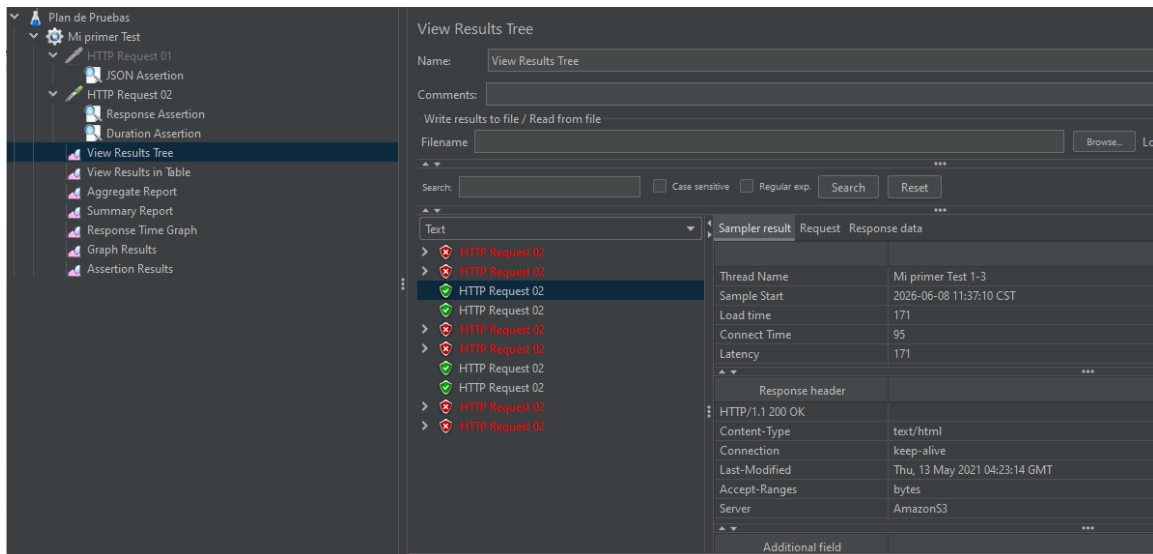
Mostrar al menos un escenario donde una aserción falle. Documentar configuración realizada, resultado esperado, resultado obtenido y explicación del fallo.



Configuración: En la petición Post_AutenticacionLogin, abrí JSON Assertion y en el cuadro de texto Expected Value puse “fail login”.

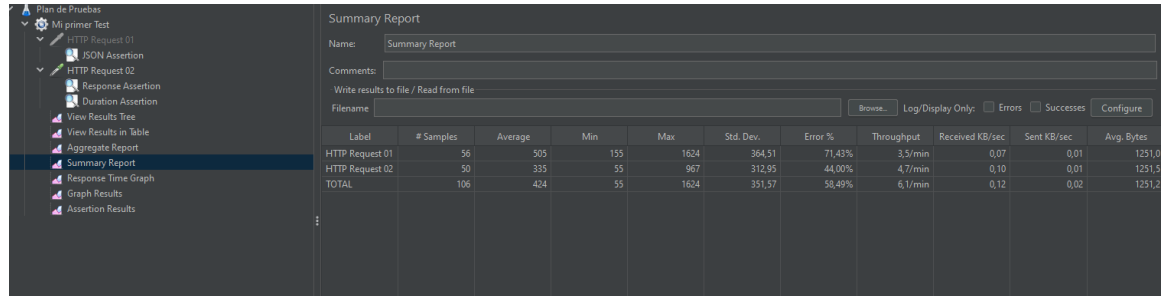
Objetivo del paso: Forzar un fallo por contenido para verificar que la herramienta lee las respuestas internas del servidor.

Resultado esperado: Que la petición falle.



Parte 7 – Análisis de Resultados

Analizar tiempo promedio de respuesta, throughput, porcentaje de errores y comportamiento observado durante cada tipo de prueba.

The screenshot shows the JMeter Summary Report interface. On the left is a tree view of the test plan. The main area displays a table with performance metrics for two HTTP requests and a total summary. The table columns include Label, # Samples, Average, Min, Max, Std. Dev., Error %, Throughput, Received KB/sec, Sent KB/sec, and Avg. Bytes. The data shows that both requests performed well with low error rates and reasonable throughput.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request 01	56	505	155	1624	364,51	71,43%	3,5/min	0,07	0,01	1251,0
HTTP Request 02	50	335	55	967	312,95	44,00%	4,7/min	0,10	0,01	1251,5
TOTAL	106	424	55	1624	351,57	58,49%	6,1/min	0,12	0,02	1251,2

Configuración: 10 usuarios, entran poco a poco duración de 5 segundos.

Resultado: Todo sale en verde, el servidor responde rapidito y hay 0% errores.

Observación: Como los usuarios entran con calma, la página trabaja de forma ideal y aguanta el tráfico normal de un día cualquiera.

Parte 8 – Documento de Evidencias – Puede ser un doc separado.

El documento debe estar elaborado como una guía paso a paso.

Para cada actividad incluir:

- Acción realizada.
- Captura de pantalla.
- Explicación de lo realizado.
- Objetivo del paso.

La documentación debe permitir que otra persona pueda reproducir la prueba / Tipo manual de usuario.

Parte 9 – GitHub

Crear un repositorio que contenga el archivo .jmx, documento de evidencias capturas utilizadas y archivos complementarios.

Entregable Final

Compartir enlace del repositorio GitHub, documento de evidencias y proyecto JMeter funcional.

Reflexión Final

Responder:

1. ¿Qué fue lo más sencillo de aprender?
2. ¿Qué fue lo más complejo?
3. ¿Qué diferencia encontró entre una prueba de carga, estrés, pico y resistencia?
4. ¿Qué aprendió sobre el uso de aserciones?
5. ¿Qué aprendió sobre el análisis de resultados en JMeter?